

Compromised — HTB SOC Sherlock Investigation

Network Traffic Analysis & Malware Forensics

Scenario

Our SOC team detected suspicious activity in network traffic. The machine has been compromised and company information that should not have been there has now been stolen – it is up to you to figure out what has happened and what data has been taken.

Task 1: Initial Access IP & Task 2: Malware Hash

Question: What is the IP address used for initial access?

Answer: 162.252.172.54

Task 2: What is the SHA256 hash of the malware?

Answer: 9b8ffdc8ba2b2caa485cca56a82b2dcdbd251f65fb30bc88f0ac3da6704e4d3c6

Methodology

Sorted IPv4 conversations to identify suspicious external IPs communicating with the victim (172.16.1.191). The IP **162.252.172.54** showed HTTP traffic downloading a file disguised as a GIF (/9GQ5A8/6ctf5JL).

The file was extracted from the pcap and submitted to VirusTotal, which flagged it as malicious. This confirmed the IP as the initial access vector.

Ethernet · 3	IPv4 · 18	IPv6	TCP · 218	UDP · 2188
Address A	Address B	Packets ▲	Bytes	
172.16.1.191	172.16.1.16	29,771	5 MB	
172.16.1.191	23.163.0.37	6,978	4 MB	
172.16.1.191	162.252.172.54	1,130	1 MB	
172.16.1.191	45.85.235.39	126	42 kB	
172.16.1.191	132.148.79.222	123	27 kB	

Figure 1: IPv4 Conversations showing suspicious IP

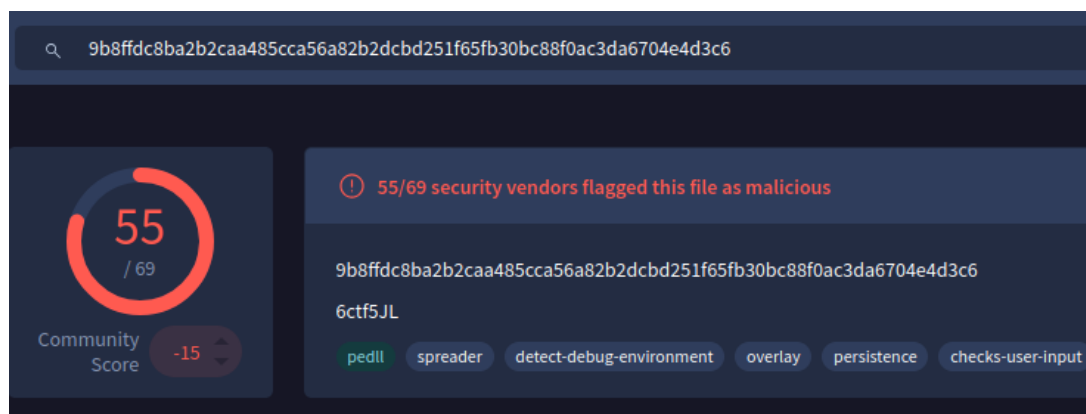


Figure 2: VirusTotal Detection - Malicious

Task 3: Malware Family

Question: What is the malware family?

Answer: pikabot

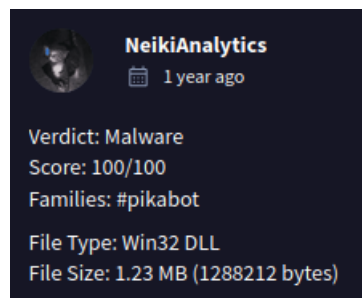


Figure 3: Malware Family: Pikabot

Task 4: First Seen Date

Question: When was the malware first seen in the wild (UTC)?

Answer: 2023-05-19 14:01:21 UTC

Methodology

The VirusTotal history panel provided the first submission timestamp for this sample.

History ⓘ	
Creation Time	2023-05-17 09:38:43 UTC
First Seen In The Wild	2023-05-19 14:01:21 UTC
First Submission	2023-05-17 19:04:23 UTC
Last Submission	2026-07-23 05:28:56 UTC
Last Analysis	2026-06-11 04:51:40 UTC

Figure 4: VirusTotal Historical Timeline

Task 5: Suspicious HTTPS Ports

Question: What are the ports used for HTTPS traffic with self-signed certificates (smallest to largest)?

Answer: 2078, 2222, 32999

Methodology

Used `tshark` to filter for TLS Certificate messages and extract the source port along with the certificate's issuer and subject details, showing only unique results:

```
tshark -r capture.pcap -Y "tls.handshake.type==11" -T fields -e tcp.srport -e x509sat.UTF8String | sort -u
```

Interpretation

- The certificate on port **443** was issued by *Comodo CA Limited* – a legitimate, globally trusted Certificate Authority. This is normal, benign HTTPS traffic.
- The certificates on ports **2078**, **2222**, and **32999** were issued by completely fictitious authorities (e.g., *"Uneared Inc."*, *"Hayburner Inc."*, *"Lapins Aerobacteriological Inc."*). They also contained nonsensical, algorithmically generated domain names like **votation.bzh** and **organizabilitybigate.rehab** – a classic malware indicator for self-signed certificates and DGA (Domain Generation Algorithm) infrastructure.

Task 6 & 7: Certificate Fields

Task 6: What is the value of `id-at-localityName`?

Answer: Pyopneumopericardium

Task 7: What is the `notBefore` timestamp (UTC)?

Answer: 2023-05-14 08:36:52 UTC

Methodology

Filtered for TLS certificates on the suspicious ports in Wirshark:

```
tls.handshake.type == 11 && (tcp.srcport == 2078 || tcp.srcport ==
    2222 || tcp.srcport == 32999)
```

```

    ... signedCertificate { ... }
    ▼ issuer: rdnSequence (0)
      ▼ rdnSequence: 6 items (id-at-commonName=votation.bzh,id-at-localityName=Py
        ▶ RDNSequence item: 1 item (id-at-countryName=SX)
        ▶ RDNSequence item: 1 item (id-at-stateOrProvinceName=KI)
        ▶ RDNSequence item: 1 item (id-at-organizationName=Uneared Inc.)
        ▶ RDNSequence item: 1 item (id-at-organizationalUnitName=Undelightful)
        ▶ RDNSequence item: 1 item (id-at-localityName=Pyopneumopericardium)
```

Figure 5: Locality Name Field

```

    ▼ signedCertificate
      version: v3 (2)
      serialNumber: 0x5651c79bfe60a17bc97bcb437c0f3ec25f7f6ec5
      ▶ signature (sha256WithRSAEncryption)
      ▶ issuer: rdnSequence (0)
      ▼ validity
        ▼ notBefore: utcTime (0)
          utcTime: 2023-05-14 08:36:52 (UTC)
        ▼ notAfter: utcTime (0)
          utcTime: 2024-05-13 08:36:52 (UTC)
```

Figure 6: notBefore Timestamp

Task 8: DNS Tunneling Domain

Question: What was the domain used for tunneling?

Answer: `steasteel.net`

Methodology

DNS traffic comprised 76.1% of the capture. Extracted all unique DNS queries:

```
tshark -r capture.pcap -Y "dns.qry.name" -T fields -e dns.qry.name |  
sort -u
```

The output showed structured queries with long hexadecimal strings under **steasteel.net** — classic DNS tunneling behavior.

Attack Chain & IP Relationships

IP Address	Role	Description
172.16.1.191	Victim	Compromised host executing Pikabot
172.16.1.16	Internal DNS	Unwitting relay forwarding tunneled queries
162.252.172.54	Attacker	Initial access IP (malware download)

Attack Flow:

1. Victim downloads malware from 162.252.172.54 (disguised as GIF).
2. Malware sends DNS queries with exfiltrated data to internal DNS (172.16.1.16).
3. Internal DNS forwards queries to the authoritative DNS server for **steasteel.net**.
4. Attacker's DNS server responds with C2 commands via DNS replies.
5. Internal DNS forwards responses back to victim.

Conclusion

The investigation successfully identified:

- Initial compromise via 162.252.172.54 delivering Pikabot malware.
- C2 communication over self-signed certificates on ports 2078, 2222, and 32999.
- Data exfiltration via DNS tunneling through **steasteel.net**.
- Internal DNS server (172.16.1.16) used as an unwitting relay.